安装说明书

这份文档有 PDF 版：[`docs/deploy.pdf`](deploy.pdf)（6 页，方便带进虚拟机照着做）。

***它是生成的**：改了本文件之后重新跑 `python scripts/md-to-docx.py docs/deploy.md x.docx` 再导出 PDF，别直接改 PDF。*

一句话：**在 Windows 上跑一条命令**，它把环境、插件、VS Code 扩展装好，然后自检。

```
.\scripts\deploy-windows.ps1
```

这份文档说明它到底做了什么、什么时候不该用它、以及出问题怎么查。命令只对 Windows 有效（插件本身不挑平台，但脚本用的 winget、junction、注册表 PATH 都是 Windows 的东西）。

零、怎么把脚本拿到机器上

三条路，第一条最快（国内一般都能通）：

```
# 1. 发布附件（推荐：内容不可变、和本地逐字节相同，改版本号即可）
irm https://github.com/wjh-ux/dsh-wechat-media/releases/download/v1.24.7/deploy-windows.ps1
-OutFile deploy.ps1


# 2. GitHub 不通时的国内前缀（慢一些，但能通）
irm https://ghproxy.net/https://github.com/wjh-ux/dsh-wechat-media/releases/download/v1.24.7/
deploy-windows.ps1 -OutFile deploy.ps1


# 3. raw 分支地址（发布后几分钟内可能还是旧内容，GitHub 的 raw 缓存）
irm https://raw.githubusercontent.com/wjh-ux/dsh-wechat-media/main/scripts/deploy-windows.ps1
-OutFile deploy.ps1
```

别用 jsDelivr 的 `@latest` / `@main`，也别指望它对刚发的标签立即可用（实测 `@v1.24.6` 发布几分钟后仍 404）。

别用 `@latest` 或 `@main`——实测它们会给你缓存的旧文件：

```
@latest    47537 字节  无版本号    ← 1.24.4 的旧内容
@main      47537 字节  无版本号    ← 同上（边缘缓存 s-maxage=43200，最长 12 小时）
@v1.24.5   48248 字节  1.24.5      ← 正确
raw        48248 字节  1.24.5      ← 正确
```

发布附件与本地文件逐字节相同（59783 字节 / 1119 行，SHA256 一致），这是"要哪一版就是哪一版"的唯一可靠路径。想要自动换路：

```
$want = '1.24.6'
$urls = @(
  "https://cdn.jsdelivr.net/gh/wjh-ux/dsh-wechat-media@v$want/scripts/deploy-windows.ps1",
  "https://raw.githubusercontent.com/wjh-ux/dsh-wechat-media/main/scripts/deploy-windows.ps1"
)
foreach ($u in $urls) {
  try { Invoke-WebRequest $u -OutFile deploy.ps1 -UseBasicParsing -TimeoutSec 30 } catch
{ continue }
  if (Select-String -Path deploy.ps1 -Pattern "\`$scriptVersion = `"$want`"" -Quiet) { "拿到 v$want
← $u"; break }
  "这条给的不是 v$want，换下一条…"
}
```

拿到脚本后，**问它自己**：

```
.\deploy.ps1 -Version
```

```
dsh-wechat-media 部署脚本 v1.24.5
 步骤总数：11
 可选功能：ToolsArchiveDir 有 / GitHubProxy / Ch340Installer 有 / SkipToolchain / SkipArduino /
SkipCh340
 说明：v1.24.3 起有 -ToolsArchiveDir；v1.24.1 起 CH340 自动装；v1.23.10 起修 pip PATH
```

（v1.24.5 之前的脚本没有这个开关，只能用下面的特征对照。）

老脚本也可以用特征对照：

```
Select-String -Path .\deploy.ps1 -Pattern 'ToolsArchiveDir' -Quiet      # True = 1.24.3 及以后
```

一、先决条件

需要	为什么	谁负责
Windows 10 1809+ / 11	脚本用 winget、目录联接、注册表 PATH	你（装系统）
DSH 本身（DeepSeek Harness）	插件是 DSH 的扩展，不是独立程序	你（装 DSH 并启动一次）
Node.js 18+	DSH 与插件工具链都要它	**脚本会装**（有 winget 用 winget，没有就下官方 msi）

Python 3.12	串口烧录依赖它	**脚本会装**（官方安装包，当前用户，带 pip）
VS Code + Python 扩展	面板/编辑器那条路	**脚本会装**（官方 User 安装包）

> ***pip 有个坑**：`pip install --user`会把`stcgal.exe`/`stm32loader.exe`放进`%APPDATA%\Python\PythonXY\Scripts`，而那个目录**通常不在 PATH 上**（pip 自己会打 WARNING 提醒）。*
>
> *插件靠`which()`找这些命令，所以脚本在第 3 步会把该目录加进**用户级** PATH —— 不加的话"装成功"和"用得上"之间差着一层。体检模式（`-CheckOnly`）与`-Offline`不会改 PATH，只会告诉你要加哪个目录。*

也就是说：一台**装了 Windows 的空机器**，只要再装好 DSH 并启动一次，剩下的脚本全包。

DSH 必须先启动过一次——它会生成 `%USERPROFILE%\.dsh\profiles\desktop\`，插件要装进那个 profile。

脚本会检查这两点，缺了会明确报 ✖ 而不是继续往下装。

> *顺序永远是：装 DSH → 启动一次 → 再跑这个脚本。*

二、一条命令做的十件事

```
[1/11] 前置条件     Windows / Node.js (缺则装) / DSH 本体 / ~/.dsh / profile
[2/11] Python 本体   没有就用 winget 或官方安装包装 Python 3.12
[3/11] Python 包     pyserial / stcgal / stm32loader (并把 pip 的 Scripts 目录加进 PATH)
[4/11] VS Code          装 VS Code + ms-python.python 扩展
[5/11] 串口驱动     CH340/CH343: 检测 (Windows Update 装的也认) , 缺了给办法
[6/11] Arduino          arduino-cli + arduino:avr 核心 (-WithEsp32Core 才装 esp32 核心)
[7/11] ARM 工具链    arm-none-eabi-gcc / cmake / ninja / openocd (xpack 便携版)
[8/11] Keil / ESP-IDF  只检测, 不安装
[9/11] 插件部署      源码 → profile 的目录联接 + package.json
[10/11] 自带 VS Code 扩展 仓库里的 dsh-wechat-media-vscode-*.vsix
[11/11] 自检        插件的解析器 + **真编一个 STM32 固件** + 冒烟测试
```

下载时会显示**进度条、已下载/总大小、实时速度、剩余时间**，结束再给一行统计：

```
    [#######------------------]  27%  6.9 MB / 25.7 MB  12.3 MB/s  还剩 2s
```

```
     25.7 MB 用时 1.8s（平均 14.2 MB/s）
```

自检不是"文件在、版本号对"就完事。第 10 步会用仓库里的 `scripts/stm32-template` **真编一个固件**：

```
✔ 工具链真能编：6 KB 的 .elf 已生成（text 212 · data 0 · bss 0）
```

这一条是吃了亏才加的：装完工具链、每个 `--version` 都对，但第一次真编译才发现问题。

它会把 **armgcc / cmake / ninja 三个 bin 目录都放进子进程 PATH**，并显式传 `-DCMAKE_MAKE_PROGRAM=<ninja.exe>`。

只加 armgcc 和 cmake 是不够的——干净机器上 ninja 不在 PATH 里，cmake 会直接报 `CMake was unable to find a build program corresponding to "Ninja"`（VM 上真实发生）。

每一步都是**先检查、再动手、最后用真命令验证**：装完 `winget` 的包会再问一次 `python --version`，装完扩展会再 `code --list-extensions --show-versions` 查一遍。文件在不在不等于能用。

没有 winget 的机器（Win10 1809 以下、精简版、部分 VM）

脚本会**退回官方安装包**，三条直链都不经过 winget：

组件	来源	静默参数
Node.js LTS	`nodejs.org/dist/<版本>/node-<版本>-x64.msi`；**MSI 失败（1603 = 不是管理员）时改用同版本的 `node-<版本>-win-x64.zip`**	`msiexec /i … /qn`；zip 走解压 + 用户 PATH
Python 3.12	`python.org/ftp/python/<版本>/python-<版本>-amd64.exe`（**从新到旧探，取第一个真的存在的**）	`/quiet InstallAllUsers=0 PrependPath=1 Include_pip=1`
VS Code	`update.code.visualstudio.com/latest/win32-x64-user/stable`（302 到真实文件）	`/VERYSILENT /NORESTART /MERGETASKS=!runcode`

先确认这台机器能连上这三个域名，用 `-ResolveOnly`（只解析地址，不下载、不安装）：

```
.\deploy.ps1 -ResolveOnly
```

```
✔ Node.js v26.13.0
  https://nodejs.org/dist/v26.13.0/node-v26.13.0-x64.msi
✔ Python 3.12.16
  https://www.python.org/ftp/python/3.12.16/python-3.12.16-amd64.exe
```

```
✔ VS Code 1.144.0
  https://vscode.download.prss.microsoft.com/…/VSCodeUserSetup-x64-1.144.0.exe
```

三个都 ✔ 就直接去掉 `-ResolveOnly` 跑；有 ✖ 说明那台机器网络到不了那个域名，把失败项的行贴出来。

Node 的 MSI 需要管理员（它是机器级安装）。VM 里如果是普通用户，会看到 `msiexec 退出码 1603`，

脚本会自动退到**免管理员的 zip**：下同版本的 `node-<版本>-win-x64.zip`，解到 `%USERPROFILE%\tools\node`，

只把它加进**用户级** PATH（不碰 HKLM）。实测那条路能装成（`v26.13.0`）。

Python 与 VS Code 本来就不需要管理员：前者 `InstallAllUsers=0`，后者是 User 安装包。

> ***Python 有个坑值得记**：python.org 的 ftp 目录里 3.12.11 及以后的文件夹**是存在的**，但那些是仅源码的安全更新，*
>
> *里面没有 `.exe`。第一版脚本取"最新目录"就拿到 `3.12.16` 的地址，下载直接 404。*
>
> *现在从新到旧逐个 HEAD，取第一个真能下的 —— 写到这份文档时是 `3.12.10`（25.7 MB，`Python 3.12.10 (64-bit)`）。*

装到哪

东西	位置
便携工具链	`%USERPROFILE%\tools\`（armgcc / cmake / ninja / openocd / arduino-cli）
插件源码	`%USERPROFILE%\dsh-wechat-media\`（或用 `-LocalPath` 指定已有检出）
进 profile 的方式	`%USERPROFILE%\.dsh\profiles\desktop\node_modules\dsh-wechat-media` 目录联接
profile 记账	`…\profiles\desktop\package.json`：`dependencies` 加 `link:`，`dsh.profile.bundles` 加名字

改 `package.json` 之前会先备份成 `package.json.bak-<时间戳>`。

三、常用写法

```
# 只体检，不改任何东西（第一次装之前建议先跑这个）
.\scripts\deploy-windows.ps1 -CheckOnly
```

```
# 完整安装
.\scripts\deploy-windows.ps1


# 只部署插件，环境已经就绪
.\scripts\deploy-windows.ps1 -SkipTools


# 用已有的检出，而不是 clone 一份
.\scripts\deploy-windows.ps1 -LocalPath D:\src\dsh-wechat-media


# 装到另一个 profile（不影响正在用的 desktop）
.\scripts\deploy-windows.ps1 -Profile staging


# 不装 Arduino（省 70 MB；只做 STC / STM32 的人可以这样）
.\scripts\deploy-windows.ps1 -SkipArduino


# 不装 ARM 工具链（省 320 MB；只想先确认插件能装进 DSH 并启动）
.\scripts\deploy-windows.ps1 -SkipToolchain


# 连 ESP32 的 Arduino 核心一起装（多 1.2 GB）
.\scripts\deploy-windows.ps1 -WithEsp32Core


# 离线：不下载、只检测并把已有东西接进 profile
.\scripts\deploy-windows.ps1 -Offline -LocalPath D:\dsh-wechat-media


# 跳过最后的冒烟测试
.\scripts\deploy-windows.ps1 -NoSelfTest
```

四、虚拟机里怎么装

VM 的问题是"什么都没有，而且网络可能慢"。两种做法：

A. 联网装（最简单）

```
# 1. 装 Node.js 18+ 和 DSH，启动 DSH 一次，然后关掉它
# 2. 拿脚本
irm https://raw.githubusercontent.com/wjh-ux/dsh-wechat-media/main/scripts/deploy-windows.ps1
-OutFile deploy.ps1
# 3. 体检
.\deploy.ps1 -CheckOnly
# 4. 安装
.\deploy.ps1
```

默认全套约 **570 MB 下载、10~20 分钟**（大头是 arm-none-eabi-gcc 320 MB）。

B. 离线装（网络差，或 VM 不许联外网）

```
# 宿主机上：把 checkout 和已经装好的工具目录一起拷进 VM
#   C:\dsh-wechat-media            ← git clone 或解压 zip
#   C:\Users\<你>\tools\           ← 宿主机上跑过一次 deploy 后的产物
# VM 里：
.\deploy-windows.ps1 -Offline -LocalPath C:\dsh-wechat-media
```

`-Offline` 的含义是：**不 clone、不 winget、不下载 zip、不装核心。**已经有的照常验证并接进 profile，缺的逐条报出来（– 行），不会卡在下载上。

四•前、GitHub 太慢时怎么办

这台机器上实测过：VS Code（微软 CDN）475 KB/s，但 **GitHub Releases 只有 1 KB/s** —— arduino-cli 17 MB 要 4.7 小时，后面还有 320 MB 的 arm-gcc。这时候给脚本一个镜像前缀：

```
.\deploy.ps1 -GitHubProxy https://ghproxy.net
```

脚本会把**只对 GitHub 的下载地址**改写成 `<前缀>/<原地址>`：

会改写的	不改写的
`github.com/*/releases/download/...` **(arduino-cli、xpack 工具链)**	`nodejs.org`（Node）
`codeload.github.com/...` **(源码 zip)**	`python.org`（Python）
`objects.githubusercontent.com/...`	`update.code.visualstudio.com`（VS Code）
	`api.github.com`（查版本号，通常不慢）

前缀末尾带不带 `/` 都行。常见的几个（哪个能用取决于你那边的网络，脚本无法替你判断）：

```
https://ghproxy.net
https://gh-proxy.com
https://ghfast.top
https://github.moeyy.xyz
```

都不行的话，两个退路：

退路一：在浏览器里把 zip 下下来，交给脚本解压（推荐）

镜像站能切镜像源，比在 PowerShell 里试前缀直观，比如 hiwiki 的 GitHub Store

（它按仓库聚合、给 SHA256、能翻历史版本）。把下好的 zip 丢进一个目录，然后：

```
.\deploy.ps1 -ToolsArchiveDir D:\downloads\tools -ToolsRoot C:\Users\<你>\tools
```

脚本按**文件名**认领（认领规则见下表），解压到 `ToolsRoot\<名字>`，解开的目录会自动提平，最后仍然跑 `--version` 验证：

zip 文件名里包含	装到	验证
arduino-cli	tools\arduino-cli	arduino-cli version
arm-none-eabi-gcc / armgcc / gnu-arm-embedded	tools\armgcc	arm-none-eabi-gcc --version
cmake	tools\cmake	cmake --version
ninja	tools\ninja	ninja --version
openocd	tools\openocd	openocd --version

已经装好的会自动跳过，所以这个开关可以反复用。**目录不存在不会算失败**——脚本提示一句"跳过，之后从网上下载"，

然后照常走在线路径（VM 上就出现过：目录没建，但从网上下载 17 MB 只用了 1.7 秒）。

下载失败会**自动重试三次**（间隔 3 秒、6 秒，每次重试前删掉半截文件），因为 无法连接到远程服务器

这类错误通常是瞬时的。三次都失败才记 ✖，并提示三条绕行路：重跑一次 / `-GitHubProxy` / `-ToolsArchiveDir`。

退路二：整体拷贝

```
# 在别的机器上跑过一次 deploy，把整个 %USERPROFILE%\tools 拷进这台机器
.\deploy.ps1 -Offline -ToolsRoot D:\tools
```

原来的两个退路：

```
# 先只要"插件能用"（跳过 320 MB 的 arm-gcc 与 70 MB 的 Arduino）
.\deploy.ps1 -SkipToolchain -SkipArduino


# 或者：在别的机器上跑过一次 deploy，把整个 %USERPROFILE%\tools 拷进这台机器，
```

```
#       再用 -Offline 指向它（脚本只做校验与联接，不下载）
.\deploy.ps1 -Offline -ToolsRoot D:\tools
```

四•补、CH340 串口驱动

STC / STM32 的板子大多用 CH340（或 CH343）做 USB 转串口。**这个驱动不用买、也不要从第三方镜像下**

（驱动能伪造签名）。两条正路：

脚本自己会装，用的是 WCH 的官方直链（这个 URL 没有扩展名，但确实返回文件）：

```
https://file.wch.cn/download/file?id=65      →  CH341SER.EXE (780 KB)
```

装之前先做两道检查，不合格就拒绝执行：

1. **是不是可执行文件** —— 头两字节必须是 `MZ`、大小 > 100 KB。CDN/WAF 有时返回 HTML 拦截页，

   那种文件验签必然失败，报错会误导人。

2. **签名** —— 必须是 `Valid`，且签名者包含 `Nanjing Qinheng Microelectronics`（WCH 的正式名称，DigiCert 签发）。

   实测：`780.4 KB`，`MZ`，签名 `Valid`，通过。

确认无误后用 `/S` 静默运行（实测该参数被接受，退出码 0），随后**再查一次** `pnputil` ——

装没装上以它为准，不看退出码。

另外两条路：插上设备并联网让 Windows Update 装（本机就是这么来的），或你手上的安装包：

```
.\deploy.ps1 -Ch340Installer D:\CH341SER.EXE   # EXE，用 /S 静默
.\deploy.ps1 -Ch340Installer D:\ch341ser.inf   # INF，用 pnputil 装
```

官方下载页（人工看说明用）：<https://www.wch.cn/downloads/CH341SER_EXE.html>

不想管这块可以跳过（不影响插件本身）：`.\deploy.ps1 -SkipCh340`。

五、装完做什么

1. **重启 DSH**（插件在启动时加载）

2. 确认在线：打开 `http://127.0.0.1:19387/dsh-wechat-media/state`，应返回带 `version` 的 JSON

3. **危险能力默认关**，用到再开：

```
/serial on      串口（烧录 STC / STM32）
/keil on        Keil 编译与烧录
/ssh on         远程执行
/task command on 定时任务里的本地命令
```

六、脚本装不了、要你自己装的两件事

	为什么	文档
Keil（C51 / MDK-ARM）	要注册账号下载，许可分商业/非商业	docs/keil-arm-install.md：下载直链、852 MB 安装包的 SHA256、器件包、四个坑
ESP-IDF	Espressif 安装器要选目标芯片与组件，交互式	脚本只检测 `export.ps1`，缺了会提示

没有它们也能用：Keil 工程之外的四种（CMake/Make、Arduino、ESP-IDF 若已装、以及纯串口烧录）照常。

六•补、DSH 起不来：Unexpected token '', "{ "n"... is not valid JSON

这是 **UTF-8 BOM**。Windows PowerShell 5.1 的 `Set-Content -Encoding utf8` 会写 BOM，而 DSH 用 `JSON.parse` 读 profile 的 `package.json`，BOM 不是合法 JSON 开头（早期版本的部署脚本就是这么写崩的）。

恢复：崩溃框上点「Disable third-party plugins, back up profile patch, and restart」先把 DSH 起来，然后去掉 BOM：

```
$f = "$env:USERPROFILE\.dsh\profiles\desktop\package.json"
$t = [IO.File]::ReadAllText($f)
if ($t.Length -gt 0 -and [int][char]$t[0] -eq 0xFEFF) { $t = $t.Substring(1) }
[IO.File]::WriteAllText($f, $t, [Text.UTF8Encoding]::new($false))
```

（v1.24.11 起脚本自己就不会写 BOM，并在写之前校验 JSON、写之后检查头三字节。）

来不及记命令的话，仓库里有个专门的修复脚本，它会报告每个 JSON 的头三字节、去掉 BOM、必要时列出 `.bak-*` 备份：

```
irm https://github.com/wjh-ux/dsh-wechat-media/releases/download/v1.24.12/repair-profile-json.ps1
-OutFile repair-profile-json.ps1
.\repair-profile-json.ps1 -CheckOnly      # 先看
.\repair-profile-json.ps1                 # 再修
.\repair-profile-json.ps1 -RestoreBackup  # 内容本身坏了才用（会丢掉插件入口）
```

七、出问题怎么查

现象	原因	怎么办
✖ 没找到 DSH 本体	DSH 没装，或装到了非默认位置	装 DSH 并启动一次；非默认位置见下面"改路径"
– 还没有 profiles	DSH 装了但没启动过	启动一次再关掉
✖ winget 装完但命令还是找不到	PATH 没刷新，或安装被取消	重新登录一次，或手动 `winget install Python.Python.3.12`
– 缺 pip 包 且装不上	python 的 `--user` 目录不可写	用 `python -m pip install --user pyserial stcgal stm32loader` 手动装看报错
✖ 那是真实目录，不是联接	`node_modules\dsh-wechat-media` 是真目录（以前手动拷过）	确认没用了再移走；脚本不会替你删
自检里 stcgal：插件找不到	装到别的 python 环境且不在 PATH	不影响插件启动；要烧 STC 就 `pip install --user stcgal`，或把它的 exe 放进 PATH
冒烟测试失败	插件代码问题	把输出的最后十几行发出来；`node test/smoke.test.mjs` 可单独跑

改路径

```
# 工具链装到别处
.\scripts\deploy-windows.ps1 -ToolsRoot D:\tools
# 插件源码放别处
.\scripts\deploy-windows.ps1 -LocalPath D:\work\dsh-wechat-media
# 装到别的 profile
.\scripts\deploy-windows.ps1 -Profile staging
```

八、卸载

插件是"一条联接 + 两行配置"，卸起来就是反过来：

```
# 1. 删联接（不要删目标目录！）
Remove-Item "$env:USERPROFILE\.dsh\profiles\desktop\node_modules\dsh-wechat-media" -Force


# 2. 从 profile 的 package.json 里去掉
#    dependencies."dsh-wechat-media" 和 dsh.profile.bundles 里的 "dsh-wechat-media"
#    改完重启 DSH
```

工具链（`%USERPROFILE%\tools`）和 pip 包留着不影响 DSH，不需要可以自行删除。

VS Code 扩展：`code --uninstall-extension wjh-ux.dsh-wechat-media-vscode`。

九、为什么脚本把「验证」写得很啰嗦

这个脚本第一版在三处栽过跟头，全部是"看起来装好了其实没用"：

1. **`Read-Version` 的第二个参数叫 `$args`** —— 那是 PowerShell 的自动变量，于是 `--version` 从来没传进去，

   每个工具的"版本"都显示成 `Usage` 或 `fatal error: no input files`。

2. **新建 profile 时只放了自己的插件** —— `bundles` 里缺 `@deepseek-ai/dsh-base` 和 `dsh-web-app`，DSH 起不来。

3. **`stcgal` 判断错** —— 它装在某台机器的 ESP-IDF python 环境里，是个独立 exe；用系统 python `import stcgal`

   会误报缺失，而实际烧录一切正常。

所以现在的判断标准是：**用插件自己的解析器（`scripts/probe-tools.mjs` 调 `which()` / `serialSupport()`）

回答它能看到什么**，而不是脚本猜 PATH。跑一遍比读十遍代码可靠。